Machine Boss: Dynamic Programming Performance Benchmarks

Machine Boss benchmark suite

March 4, 2026

1 Methodology

We benchmark the Forward and Viterbi dynamic programming algorithms across multiple backends in the Machine Boss WFST toolkit:

- **C++ interpreter**: the `boss` CLI generic DP engine (`-L` for Forward, `-V` for Viterbi), using hash-map lookups for transitions
- **C++ compiled**: machine-specific C++ code generated by `boss --cpp64 --codegen`, compiled with `clang++ -O3`; uses direct array access instead of hash-map lookups
- **JAX 1D scan**: Python/JAX using `jax.lax.scan` (sequential)
- **JAX 1D parallel**: Python/JAX using `jax.lax.associative_scan` (parallel prefix)
- **JAX 2D scan**: Python/JAX 2D with sequential outer scan
- **JAX 2D wavefront**: Python/JAX 2D with anti-diagonal wavefront parallelism
- **JAX GPU**: JAX on GPU (skipped if unavailable)
- **JS CPU**: JavaScript CPU fallback via Node.js

1.1 1D problems (generators)

Random generator machines (output-only) with DNA alphabet (A, C, G, T) are tested across:

- Sequence length $L \in \{100, 500, 2000, 10000\}$
- Machine states $S \in \{4, 16, 64\}$

1.2 2D problems (transducers)

Random transducer machines with DNA alphabet are tested across:

- Input length $L_{\text{in}} \in \{10, 50, 200\}$
- Output length $L_{\text{out}} \in \{10, 50, 200\}$
- Machine states $S \in \{4, 16, 64\}$

Each configuration is timed over multiple repetitions after a warmup call (which also triggers JIT compilation for JAX backends). We report mean $\pm$ standard deviation in seconds. Configurations that exceed a timeout threshold are recorded with their probe timing and larger sizes are skipped.

2 Hardware

Property	Value
wideboy.local / Intel(R) Xeon(R) W-3223 CPU @ 3.50GHz	
System	Darwin wideboy.local 24.1.0 Darwin Kernel Version 24.1.0: Thu Oct 10 21:02:27 PDT 2024; root:xnu-11215.41.3~2/RELEASE_X86_64 x86_64
CPU	Intel(R) Xeon(R) W-3223 CPU @ 3.50GHz
Physical cores	8
Logical cores	16
CPU frequency	3.50 GHz
Memory	32 GB
Python	3.12.6
Platform	macOS-15.1.1-x86_64-i386-64bit

3 Results: 1D Forward

Table 1: 1D Forward timings (generators) (seconds, mean ± std).

S	L	`cpp_compiled`	`cpp_interp`	`jax_1d_optimal`	`jax_1d_simple`	`js_cpu`
4	100	0.0021 ± 0.0002	0.048 ± 0.014	0.089 ± 0.006	0.153 ± 0.015	0.0019 ± 0.0007
4	500	0.0020 ± 0.0001	0.036 ± 0.000	0.115 ± 0.015	0.167 ± 0.015	0.0042 ± 0.0029
4	2000	0.0021 ± 0.0001	0.037 ± 0.001	0.105 ± 0.004	0.185 ± 0.002	0.0100 ± 0.0058
4	10000	0.0022 ± 0.0000	0.041 ± 0.000	0.144 ± 0.006	0.215 ± 0.014	0.034 ± 0.006
16	100	0.0022 ± 0.0002	0.036 ± 0.000	0.125 ± 0.011	0.192 ± 0.026	0.0061 ± 0.0052
16	500	0.0021 ± 0.0001	0.036 ± 0.000	0.147 ± 0.021	0.271 ± 0.018	0.016 ± 0.007
16	2000	0.0021 ± 0.0000	0.039 ± 0.000	0.278 ± 0.019	0.228 ± 0.016	0.051 ± 0.008
16	10000	0.0028 ± 0.0002	0.053 ± 0.001	1.03 ± 0.07	0.378 ± 0.013	0.246 ± 0.014
64	100	0.0020 ± 0.0000	0.039 ± 0.000	0.492 ± 0.061	0.257 ± 0.016	0.039 ± 0.010
64	500	0.0019 ± 0.0000	0.041 ± 0.000	2.78 ± 0.14	0.358 ± 0.029	0.156 ± 0.007
64	2000	0.0025 ± 0.0000	0.051 ± 0.000	15.38 ± 1.00	0.868 ± 0.038	0.579 ± 0.017
64	10000	0.0042 ± 0.0001	0.101 ± 0.001	200.65 ± 0.00	2.77 ± 0.02	2.79 ± 0.03

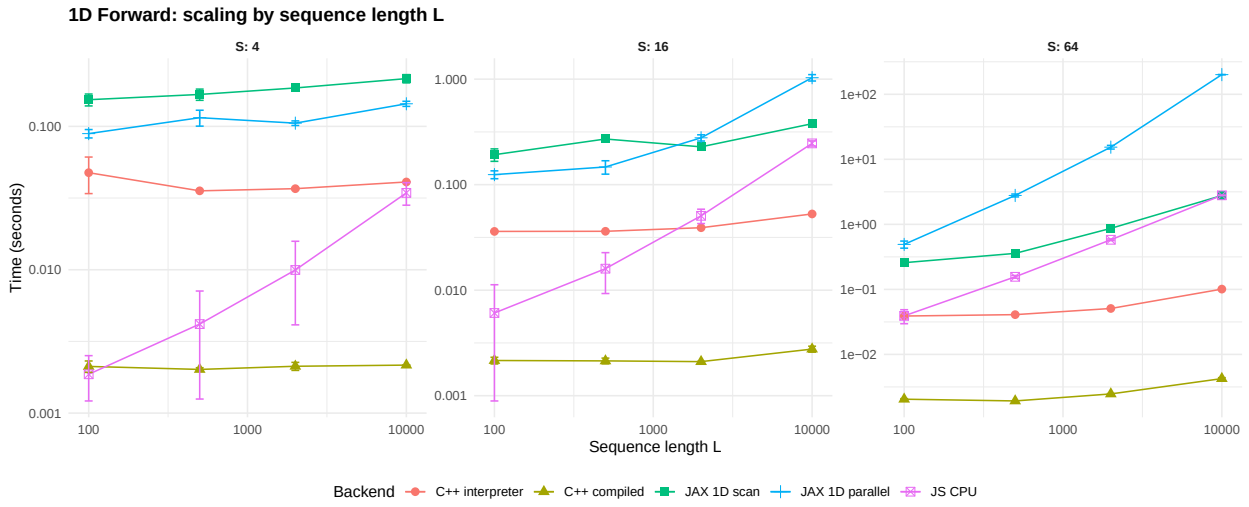

Figure 1: 1D Forward: scaling by sequence length L, faceted by states S. Log-log scale.

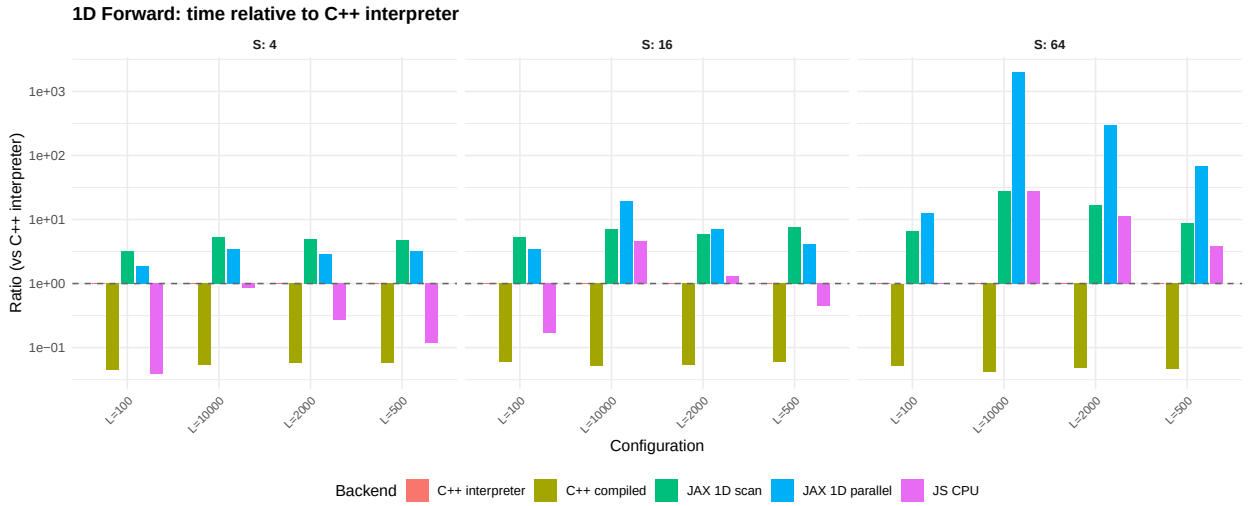

Figure 2: 1D Forward: time relative to C++ interpreter, by configuration.

4 Results: 1D Viterbi

Table 2: 1D Viterbi timings (generators) (seconds, mean ± std).

S	L	cpp_compiled	cpp_interp	jax_1d_optimal	jax_1d_simple	js_cpu
4	100	0.0019 ± 0.0000	0.035 ± 0.000	0.050 ± 0.004	0.101 ± 0.008	0.0015 ± 0.0003
4	500	0.0022 ± 0.0002	0.037 ± 0.001	0.053 ± 0.003	0.103 ± 0.002	0.0040 ± 0.0027
4	2000	0.0021 ± 0.0001	0.042 ± 0.000	0.063 ± 0.005	0.100 ± 0.009	0.0095 ± 0.0039
4	10000	0.0021 ± 0.0000	0.065 ± 0.000	0.072 ± 0.003	0.126 ± 0.003	0.030 ± 0.006
16	100	0.0020 ± 0.0000	0.036 ± 0.000	0.058 ± 0.008	0.121 ± 0.007	0.0065 ± 0.0039
16	500	0.0020 ± 0.0000	0.038 ± 0.000	0.058 ± 0.003	0.109 ± 0.009	0.016 ± 0.007
16	2000	0.0022 ± 0.0002	0.044 ± 0.000	0.109 ± 0.003	0.152 ± 0.008	0.051 ± 0.005
16	10000	0.0026 ± 0.0002	0.075 ± 0.000	0.354 ± 0.003	0.182 ± 0.013	0.239 ± 0.006
64	100	0.0020 ± 0.0000	0.039 ± 0.000	0.218 ± 0.005	0.269 ± 0.118	0.036 ± 0.005
64	500	0.0020 ± 0.0000	0.043 ± 0.000	0.939 ± 0.030	0.170 ± 0.010	0.156 ± 0.007
64	2000	0.0021 ± 0.0000	0.055 ± 0.000	4.90 ± 0.04	0.270 ± 0.018	0.581 ± 0.014
64	10000	0.0027 ± 0.0002	0.124 ± 0.002	29.61 ± 1.97	1.15 ± 0.04	2.84 ± 0.06

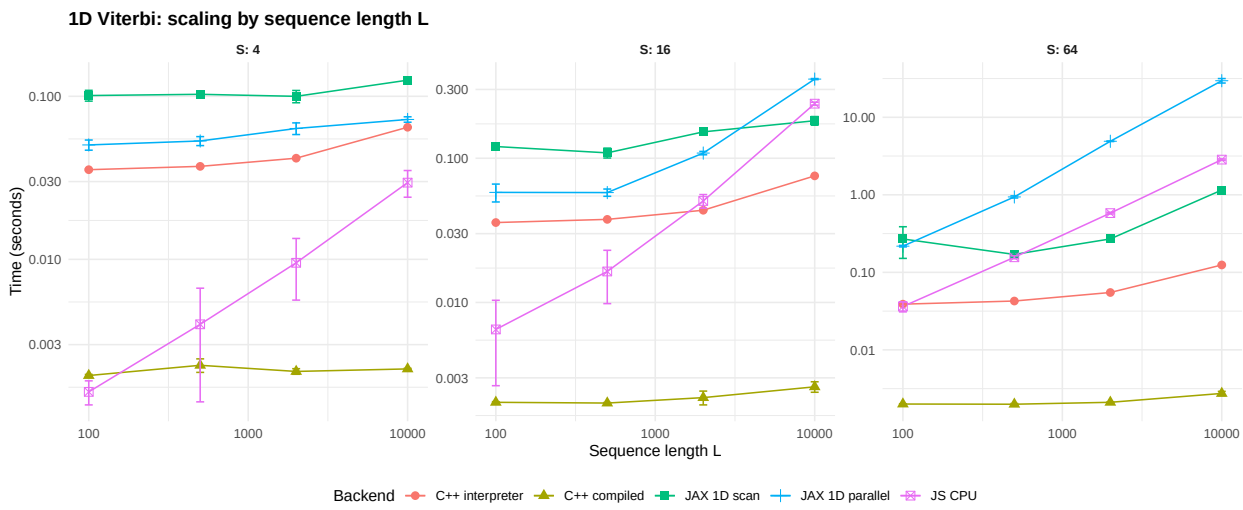

Figure 3: 1D Viterbi: scaling by sequence length L, faceted by states S. Log-log scale.

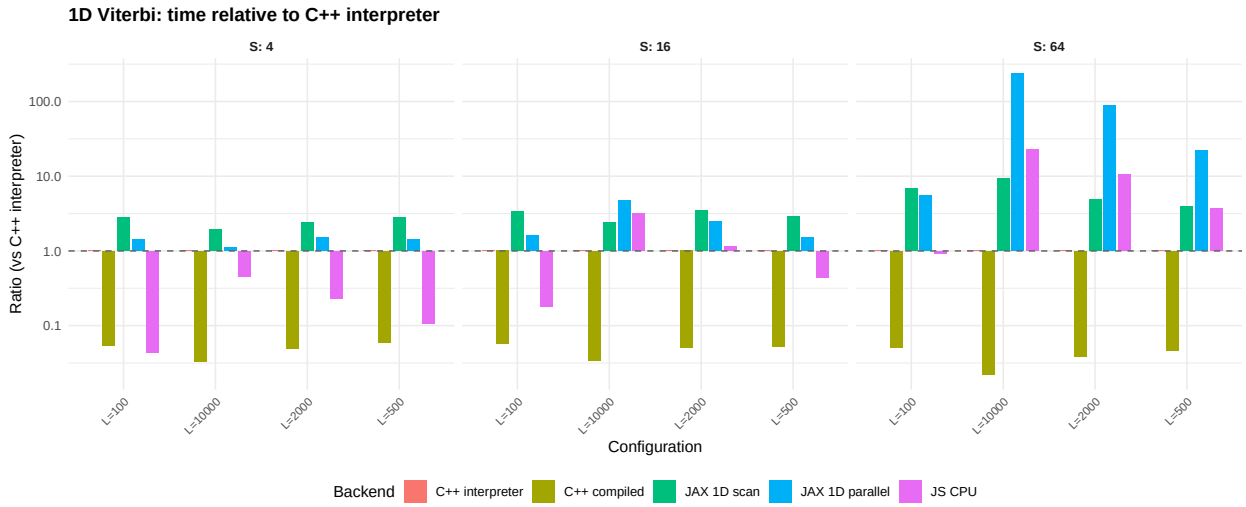

Figure 4: 1D Viterbi: time relative to C++ interpreter, by configuration.

5 Results: 2D Forward

Table 3: 2D Forward timings (transducers) (seconds, mean ± std).

S	L_{in}	L_{out}	cpp_compiled	cpp_interp	jax_2d_optimal	jax_2d_simple	js_cpu
4	10	10	0.0022 ± 0.0002	0.037 ± 0.001	0.431 ± 0.030	0.600 ± 0.036	0.0021 ± 0.0004
4	10	50	0.0020 ± 0.0000	0.036 ± 0.001	0.426 ± 0.011	0.961 ± 0.392	0.0043 ± 0.0024
4	10	200	0.0030 ± 0.0003	0.037 ± 0.000	0.442 ± 0.058	0.770 ± 0.048	0.014 ± 0.003
4	50	10	0.0020 ± 0.0001	0.036 ± 0.000	0.393 ± 0.013	0.744 ± 0.178	0.0050 ± 0.0032
4	50	50	0.0020 ± 0.0000	0.038 ± 0.000	0.432 ± 0.040	0.782 ± 0.035	0.015 ± 0.003
4	50	200	0.0022 ± 0.0001	0.045 ± 0.004	0.463 ± 0.051	0.884 ± 0.030	0.053 ± 0.005
4	200	10	0.0020 ± 0.0000	0.038 ± 0.000	0.416 ± 0.033	0.579 ± 0.014	0.013 ± 0.003
4	200	50	0.0022 ± 0.0002	0.041 ± 0.000	0.466 ± 0.021	0.850 ± 0.051	0.052 ± 0.005
4	200	200	0.0027 ± 0.0002	0.055 ± 0.000	0.597 ± 0.016	1.10 ± 0.12	0.191 ± 0.010
16	10	10	0.0020 ± 0.0000	0.039 ± 0.000	0.466 ± 0.054	0.710 ± 0.068	0.0069 ± 0.0035
16	10	50	0.0020 ± 0.0000	0.039 ± 0.000	0.478 ± 0.014	0.961 ± 0.041	0.024 ± 0.004
16	10	200	0.0022 ± 0.0000	0.042 ± 0.000	0.503 ± 0.049	1.14 ± 0.13	0.086 ± 0.004
16	50	10	0.0020 ± 0.0001	0.039 ± 0.000	0.580 ± 0.081	0.829 ± 0.085	0.027 ± 0.006
16	50	50	0.0021 ± 0.0000	0.043 ± 0.000	0.543 ± 0.024	1.05 ± 0.05	0.105 ± 0.007
16	50	200	0.0026 ± 0.0000	0.057 ± 0.000	0.622 ± 0.054	1.64 ± 0.06	0.403 ± 0.004
16	200	10	0.0023 ± 0.0002	0.042 ± 0.000	0.494 ± 0.044	1.00 ± 0.02	0.086 ± 0.004
16	200	50	0.0026 ± 0.0001	0.061 ± 0.006	0.582 ± 0.026	1.77 ± 0.07	0.398 ± 0.009
16	200	200	0.0043 ± 0.0001	0.112 ± 0.007	1.27 ± 0.05	2.96 ± 0.14	1.52 ± 0.01
64	10	10	0.0022 ± 0.0002	0.049 ± 0.000	0.567 ± 0.027	1.16 ± 0.04	0.065 ± 0.009
64	10	50	0.0022 ± 0.0000	0.051 ± 0.000	0.639 ± 0.046	2.02 ± 0.03	0.279 ± 0.010
64	10	200	0.0026 ± 0.0001	0.064 ± 0.000	0.859 ± 0.057	5.18 ± 0.24	1.09 ± 0.02
64	50	10	0.0022 ± 0.0000	0.052 ± 0.000	0.670 ± 0.014	2.08 ± 0.04	0.282 ± 0.012
64	50	50	0.0027 ± 0.0000	0.068 ± 0.000	1.02 ± 0.04	5.71 ± 0.11	1.30 ± 0.04
64	50	200	0.0050 ± 0.0001	0.127 ± 0.000	2.29 ± 0.08	18.84 ± 0.34	5.19 ± 0.04
64	200	10	0.0027 ± 0.0001	0.064 ± 0.000	0.974 ± 0.026	5.33 ± 0.13	1.08 ± 0.04
64	200	50	0.0049 ± 0.0002	0.126 ± 0.000	2.46 ± 0.41	19.40 ± 0.25	5.18 ± 0.08
64	200	200	0.013 ± 0.000	0.358 ± 0.001	9.95 ± 0.04	70.69 ± 0.00	21.72 ± 0.34

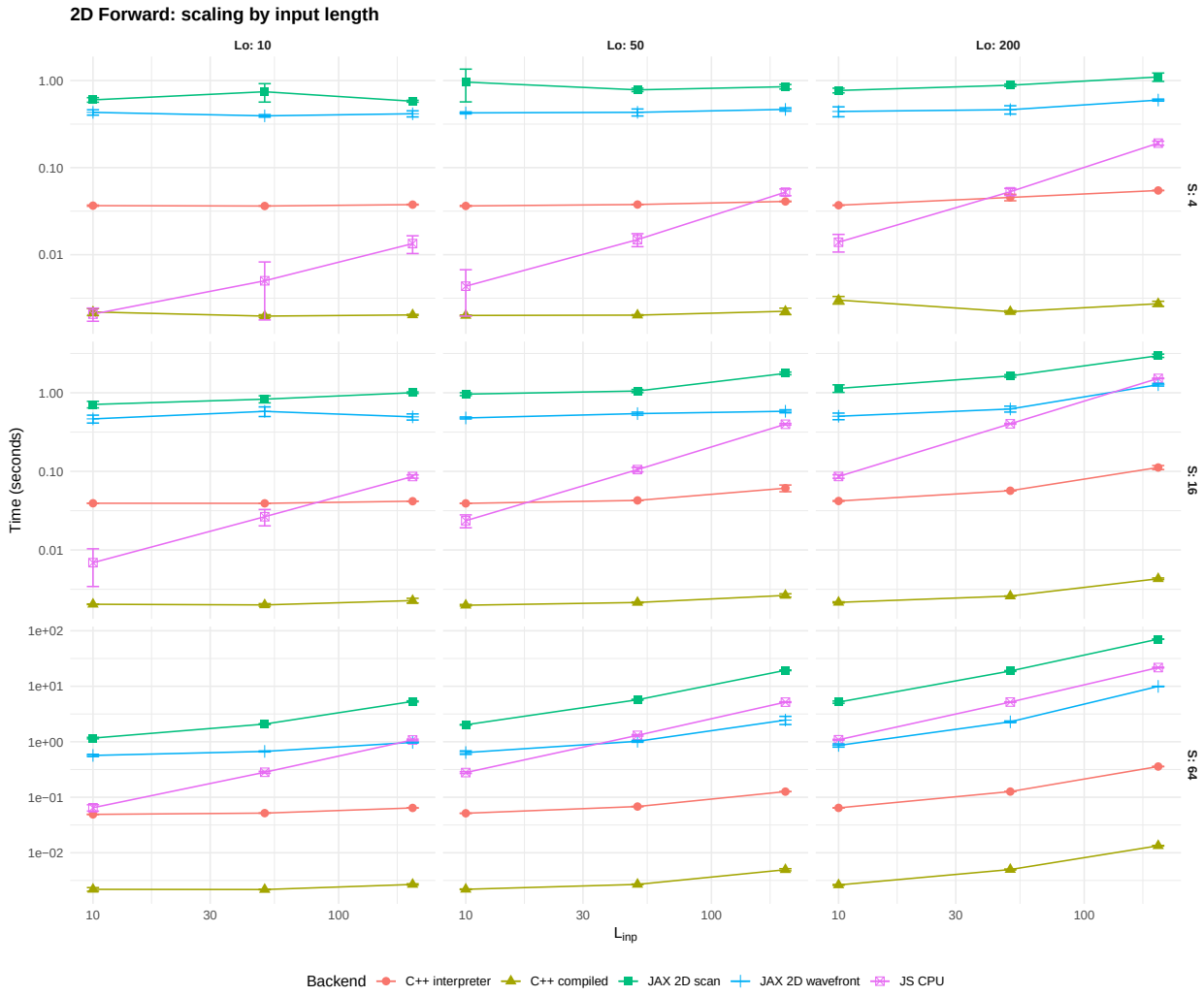

Figure 5: 2D Forward: scaling by input length L_{in}, faceted by states S (rows) and output length L_{out} (columns). Log-log scale.

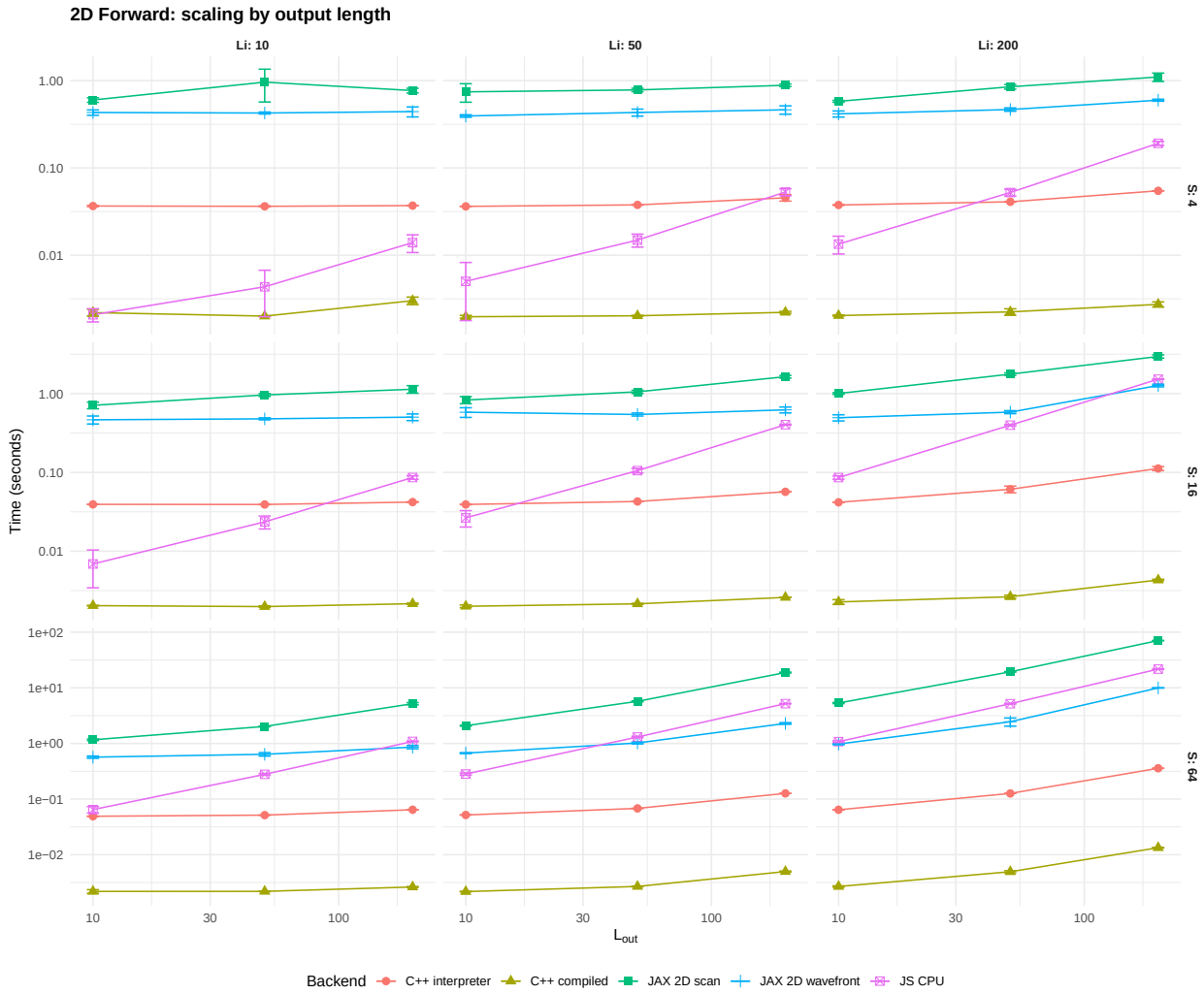

Figure 6: 2D Forward: scaling by output length L_{out}, faceted by states S (rows) and input length L_{in} (columns). Log-log scale.

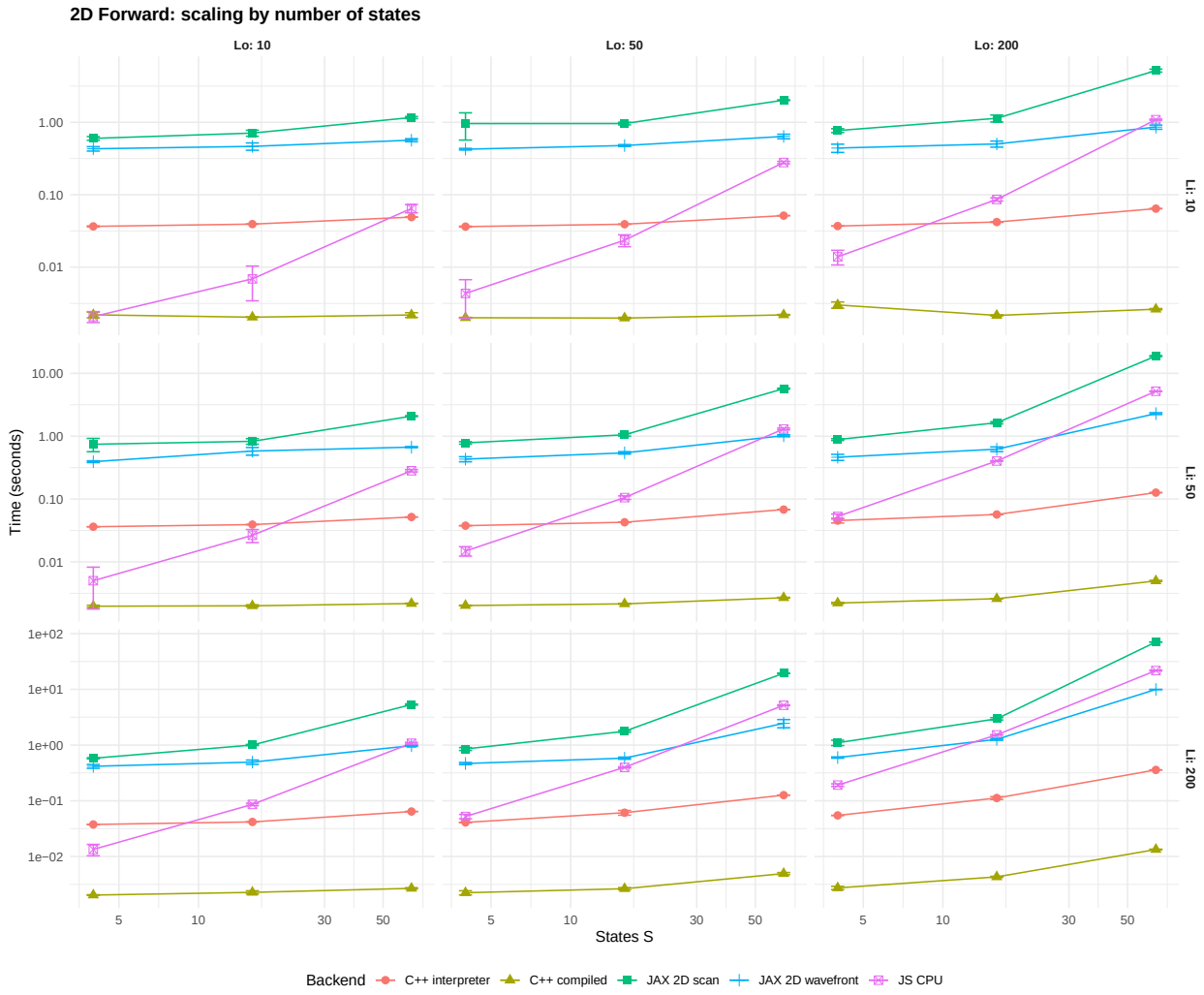

Figure 7: 2D Forward: scaling by states S, faceted by L_{in} (rows) and L_{out} (columns). Log-log scale.

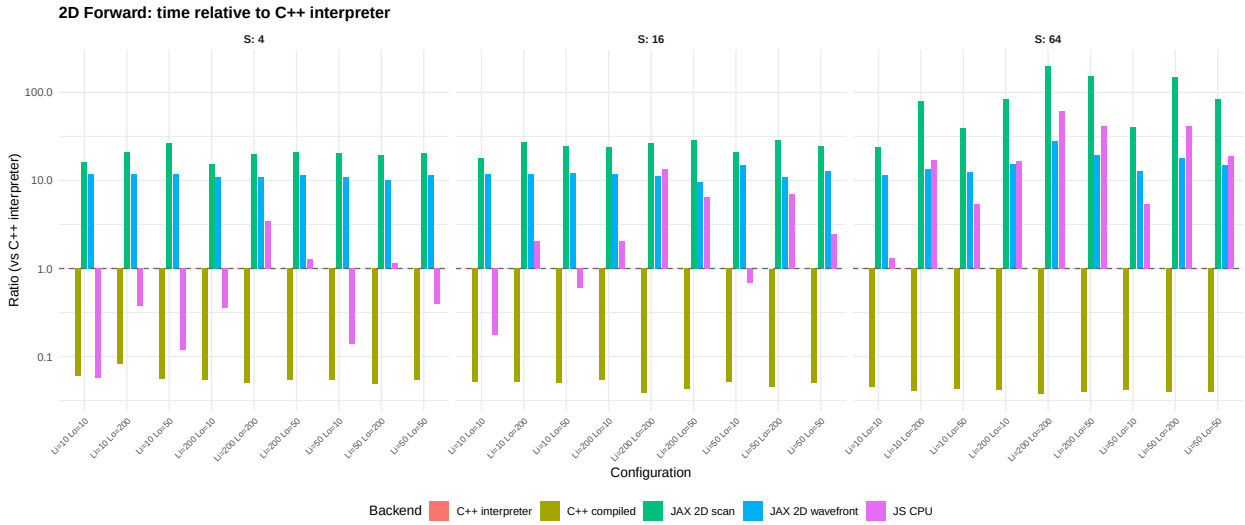

Figure 8: 2D Forward: time relative to C++ interpreter, by configuration.

6 Results: 2D Viterbi

Table 4: 2D Viterbi timings (transducers) (seconds, mean ± std).

S	L_{in}	L_{out}	cpp_compiled	cpp_interp	jax_2d_optimal	jax_2d_simple	js_cpu
4	10	10	0.0020 ± 0.0000	0.036 ± 0.000	0.314 ± 0.024	0.344 ± 0.040	0.0015 ± 0.0003
4	10	50	0.0021 ± 0.0002	0.036 ± 0.000	0.329 ± 0.030	0.407 ± 0.046	0.0039 ± 0.0017
4	10	200	0.0022 ± 0.0002	0.037 ± 0.000	0.272 ± 0.011	0.435 ± 0.016	0.013 ± 0.002
4	50	10	0.0021 ± 0.0001	0.037 ± 0.000	0.310 ± 0.009	0.324 ± 0.010	0.0038 ± 0.0017
4	50	50	0.0022 ± 0.0001	0.038 ± 0.000	0.302 ± 0.030	0.406 ± 0.027	0.015 ± 0.002
4	50	200	0.0021 ± 0.0000	0.041 ± 0.000	0.300 ± 0.035	0.634 ± 0.242	0.055 ± 0.010
4	200	10	0.0021 ± 0.0002	0.037 ± 0.000	0.329 ± 0.025	0.369 ± 0.040	0.013 ± 0.002
4	200	50	0.0020 ± 0.0000	0.040 ± 0.000	0.298 ± 0.025	0.425 ± 0.049	0.049 ± 0.006
4	200	200	0.0024 ± 0.0001	0.060 ± 0.005	0.380 ± 0.025	0.808 ± 0.271	0.180 ± 0.012
16	10	10	0.0020 ± 0.0000	0.038 ± 0.000	0.294 ± 0.018	0.429 ± 0.019	0.0062 ± 0.0027
16	10	50	0.0022 ± 0.0001	0.039 ± 0.001	0.304 ± 0.020	0.431 ± 0.037	0.022 ± 0.003
16	10	200	0.0020 ± 0.0000	0.042 ± 0.000	0.320 ± 0.019	0.532 ± 0.038	0.081 ± 0.004
16	50	10	0.0024 ± 0.0002	0.042 ± 0.001	0.319 ± 0.040	0.400 ± 0.040	0.023 ± 0.004
16	50	50	0.0020 ± 0.0001	0.043 ± 0.000	0.326 ± 0.032	0.594 ± 0.031	0.097 ± 0.005
16	50	200	0.0023 ± 0.0000	0.056 ± 0.000	0.336 ± 0.008	0.695 ± 0.041	0.371 ± 0.007
16	200	10	0.0020 ± 0.0000	0.042 ± 0.000	0.319 ± 0.035	0.480 ± 0.017	0.081 ± 0.005
16	200	50	0.0023 ± 0.0000	0.055 ± 0.001	0.335 ± 0.020	0.870 ± 0.052	0.371 ± 0.008
16	200	200	0.0030 ± 0.0000	0.105 ± 0.001	0.660 ± 0.007	1.38 ± 0.06	1.41 ± 0.03
64	10	10	0.0021 ± 0.0001	0.049 ± 0.001	0.358 ± 0.043	0.579 ± 0.052	0.061 ± 0.004
64	10	50	0.0021 ± 0.0001	0.051 ± 0.001	0.394 ± 0.045	1.05 ± 0.08	0.272 ± 0.008
64	10	200	0.0025 ± 0.0002	0.065 ± 0.001	0.507 ± 0.022	2.71 ± 0.09	1.03 ± 0.04
64	50	10	0.0020 ± 0.0000	0.053 ± 0.001	0.376 ± 0.003	1.01 ± 0.07	0.273 ± 0.007
64	50	50	0.0025 ± 0.0001	0.068 ± 0.000	0.496 ± 0.037	2.99 ± 0.05	1.22 ± 0.02
64	50	200	0.0032 ± 0.0002	0.124 ± 0.001	0.950 ± 0.009	10.28 ± 0.15	4.88 ± 0.07
64	200	10	0.0024 ± 0.0001	0.064 ± 0.000	0.505 ± 0.031	2.74 ± 0.05	1.02 ± 0.02
64	200	50	0.0033 ± 0.0003	0.124 ± 0.000	0.866 ± 0.004	10.60 ± 0.15	4.90 ± 0.07
64	200	200	0.0063 ± 0.0000	0.350 ± 0.009	3.74 ± 0.04	39.05 ± 0.06	19.55 ± 0.16

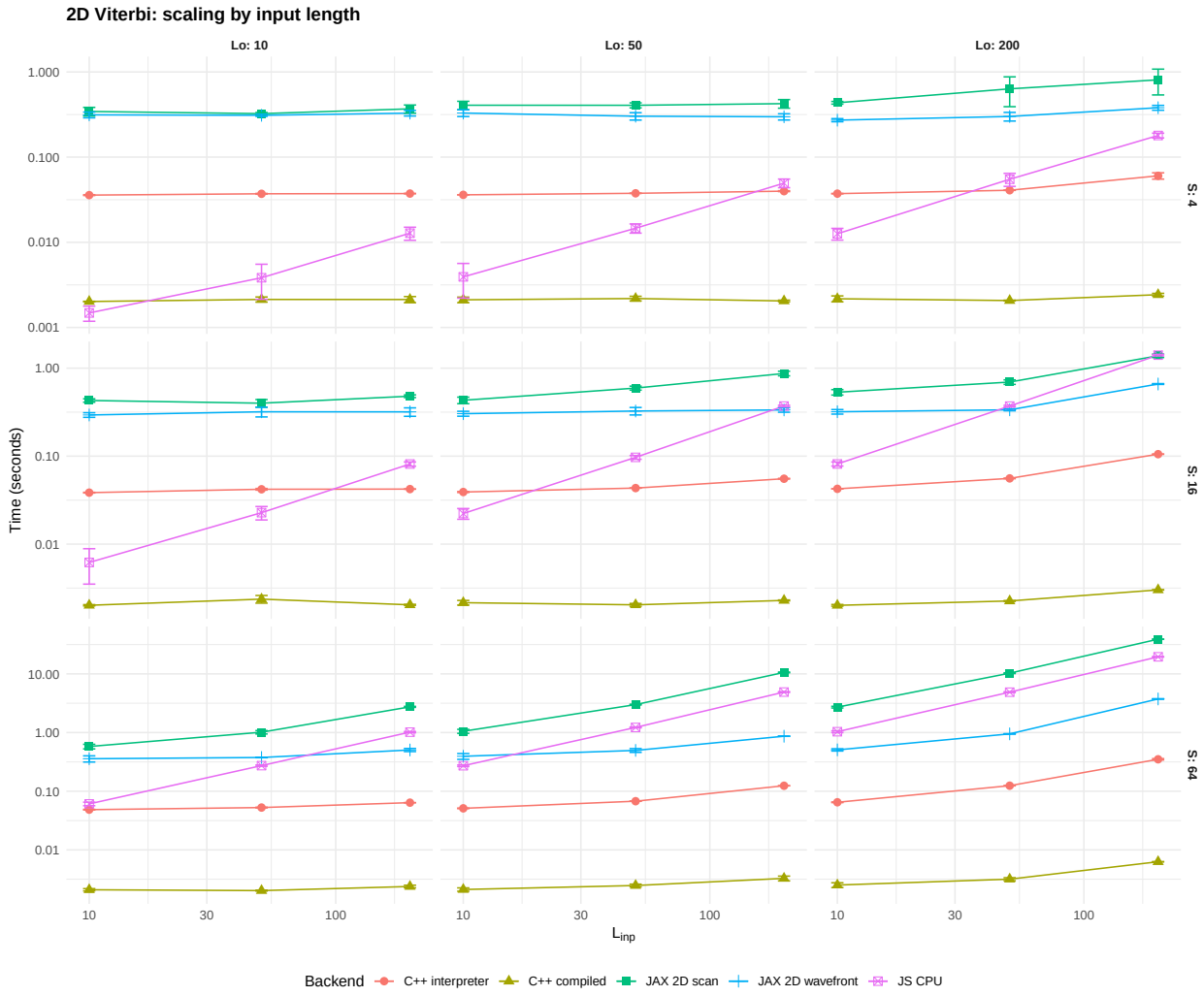

Figure 9: 2D Viterbi: scaling by input length L_{in}, faceted by states S (rows) and output length L_{out} (columns). Log-log scale.

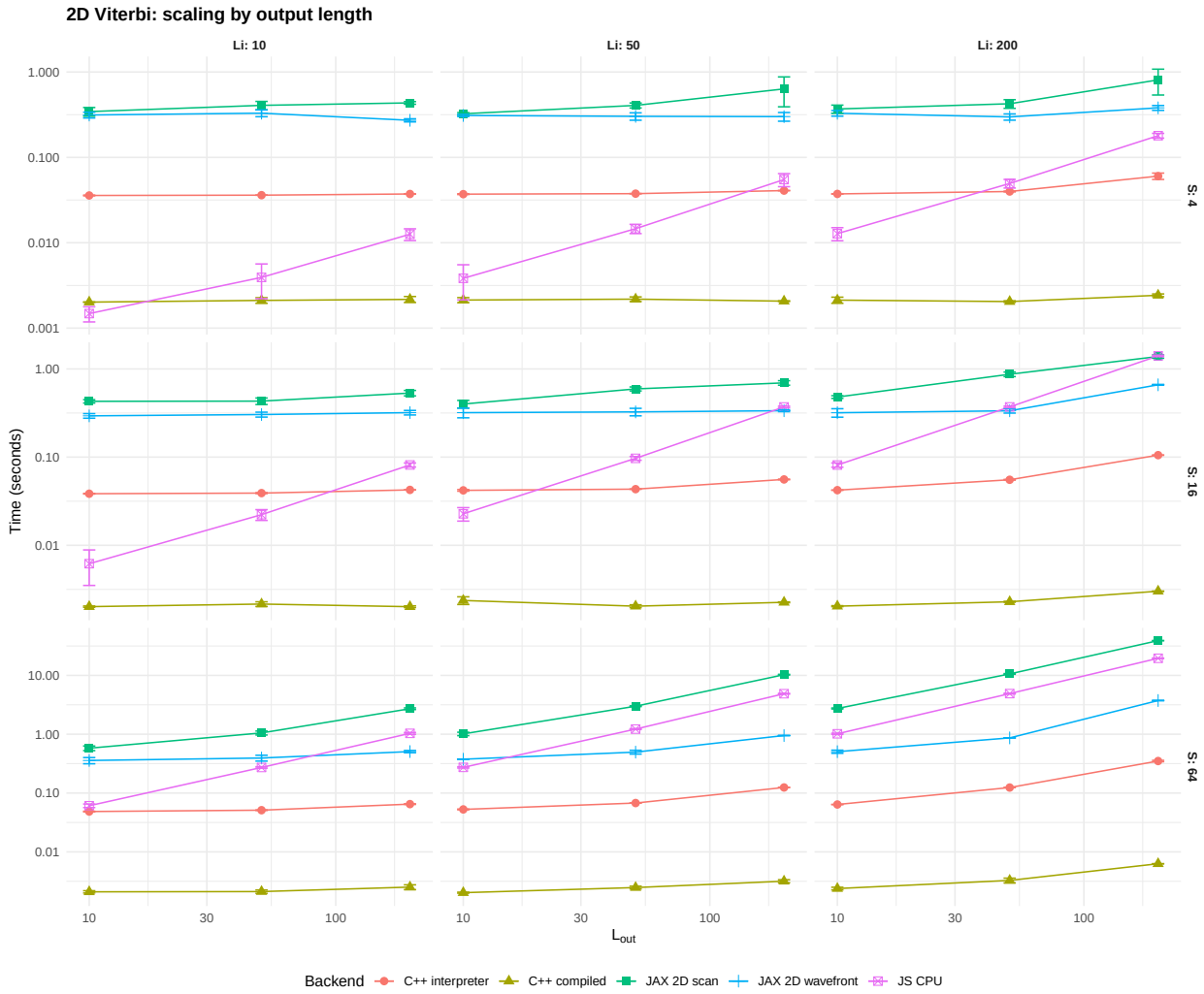

Figure 10: 2D Viterbi: scaling by output length L_{out}, faceted by states S (rows) and input length L_{in} (columns). Log-log scale.

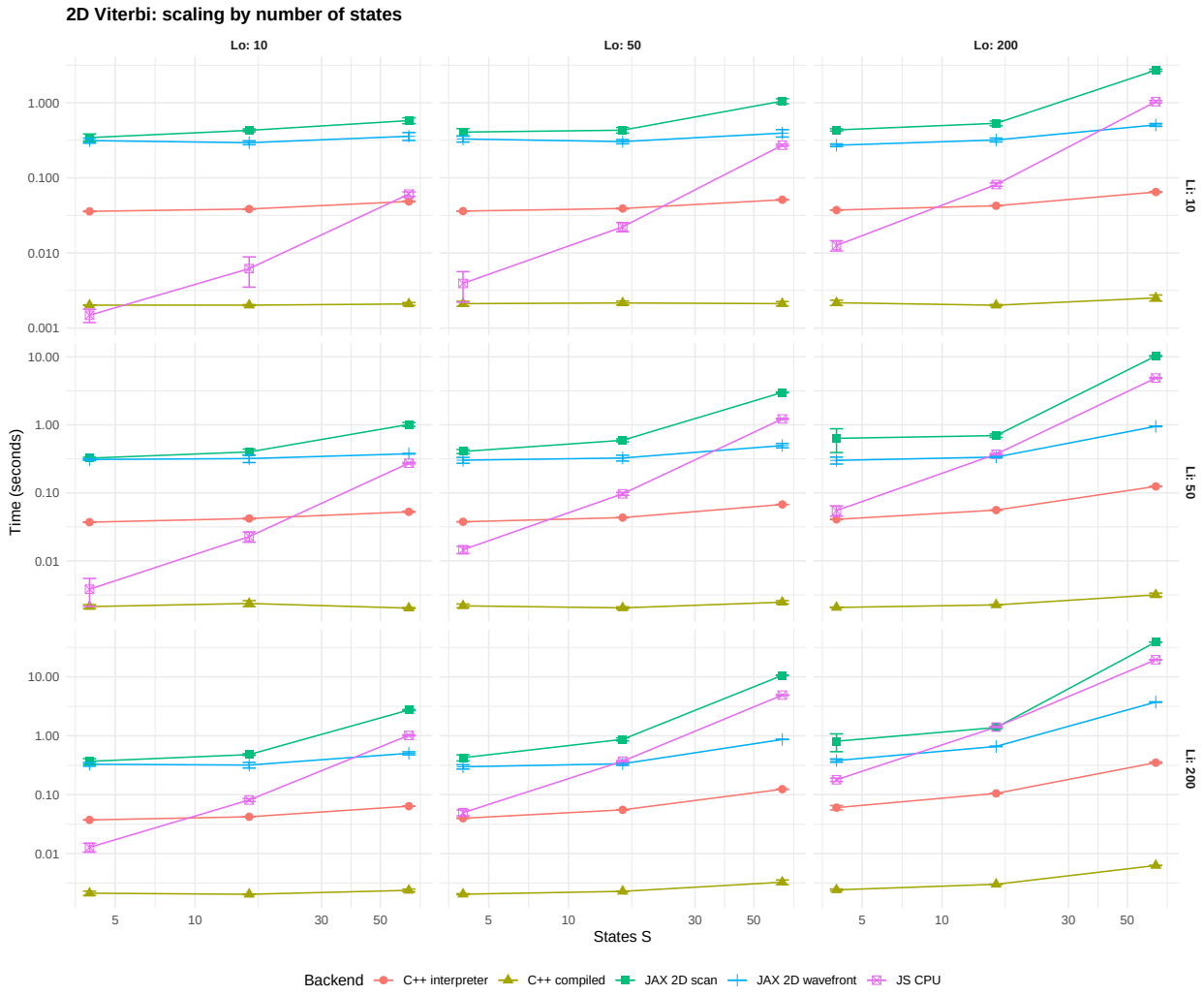

Figure 11: 2D Viterbi: scaling by states S, faceted by L_{in} (rows) and L_{out} (columns). Log-log scale.

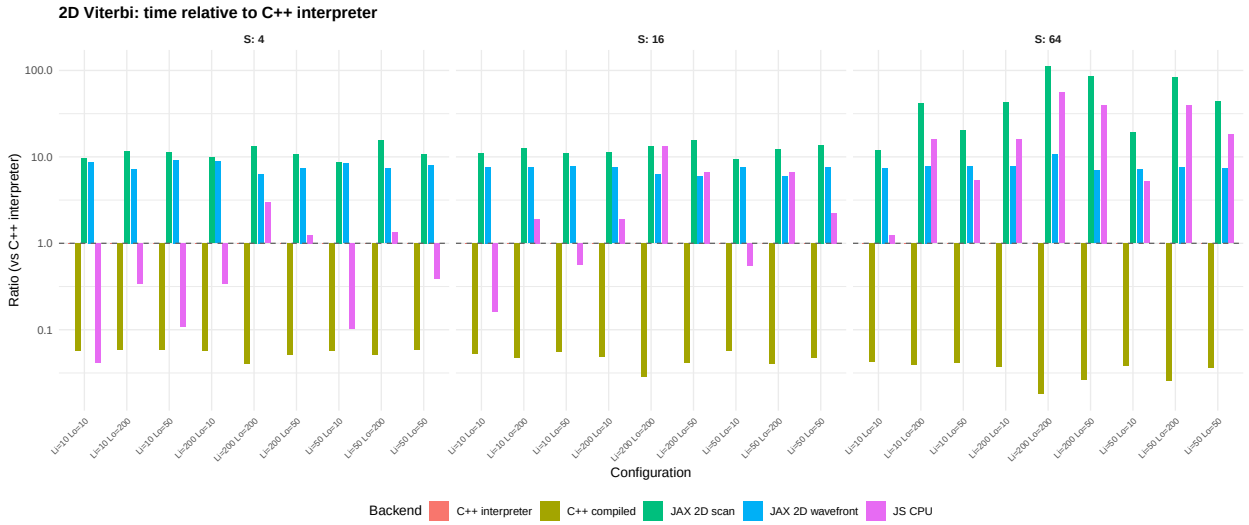

Figure 12: 2D Viterbi: time relative to C++ interpreter, by configuration.